# Testování softwaru

**Přednášky 6, 7, 8**:
Jendotkové testování:
JUnit, TestNG a základy efektivního návrhu

Miroslav Bureš 2016

# Základní principy a JUnit

# Unit testy

- Odhalují chyby na úrovni kódu

- **Princip unit testu**:
  - Srovnání výsledku volání metody / procedury s očekávaným výsledkem
  - K programu vytvořím další část kódu, který kontroluje „funkční část“ programu

- Takovou kontrolu může programátor napsat ke svému kódu sám
- Unit test framework k tomu dává podporu a šetří čas na přípravu takových testů

- Unit testy je možné **opakovaně spouštět po každé změně programu**
- Vhodné zapracovat jako součást build procedury

- Základní nástroj pro Test Driven Development – zde plní roli specifikace výsledku (viz dále)

# Výhody a limity unit testů



| Výhody | Limity |
| --- | --- |
| • Účinná metoda proti zavlečení regresních chyb<br>• Příprava jednou, běh prakticky zdarma<br>• Odchytím řadu chyb spojených s pády programu → nezdržuji testery těmito chybami | • Není ekvivalentní náhrada za funkční nebo uživatelské testy<br>• Možná iluze, že velký počet unit testů automaticky znamená účinné otestování |

# Unit test framework



- Je jich vytvořená řada
  - http://en.wikipedia.org/wiki/List_of_unit_testing_frameworks

- Pro Javu je nejznámější **JUnit**: http://junit.org/
  - Součástí instalace nových verzí vývojových prostředí (např. NetBeans)
  - Popis API je na http://junit.sourceforge.net/javadoc/

- Dále se budeme zabývat JUnit:
  - Kód a části test frameworku budou Java a JUnit
  - Principy ale platí obecně a ostatní xUnit frameworky jsou podobné

# Organizace unit testů



- Zvláštní sada balíčků s třídami unit testů, zařazené v projektu jako test packages

- **Třída unit testu**:
  - Sada metod pro inicializaci a vyčištění prostředí před a po běhu testů
  - Sada testovacích metod, vyhodnocení výsledků testů pomocí metod typu assert (viz dále)

- Není vynucené přímé mapování „konkrétní třída kódu – konkrétní unit test“ a „metoda testované třídy – metoda unit testu“
- Taková organizace je ale pro většinu případů vhodná (viz dále)

# Struktura JUnit testu

| **Anotace**<br><br>v JUnit 4 | **Název metody**<br><br>Původně v JUnit 3<br>Pro JUnit 4 konvence | **Popis** |
|---|---|---|
| @BeforeClass<br>*statická* | setUpClass() | • Spuštěna na začátku testu třídy<br>• Nastavení části programu do stavu potřebného pro testy:<br>*Inicializace připojení k DB, nastavení testovacích dat, …* |
| @AfterClass<br>*statická* | tearDownClass() | • Spuštěna na konci testu třídy<br>• „Úklid po testech“, aby byla aplikace ve použitelném stavu pro test další třídy:<br>*Zavření spojení do databáze, smazání pomocných dat, …* |
| @Before | setUp() | • Spuštěna na začátku každého testu |
| @After | tearDown() | • Spuštěna na konci každého testu |
| @Test | testNazevMetody() | • Metoda, která provádí test |

# Sekvence metod

Běh JUnit testu:

@BeforeClass
- konstruktor
- @Before
  - **@Test1**
- @After
- konstruktor
- @Before
  - **@Test2**
- @After
- konstruktor
- @Before
  - **@Test3**
- @After

@AfterClass

- **@Before** a **@After** jsou pro všechny **@Test** metody dané třídy unittestu společné. Pokud je pro konkrétní testovací metodu potřeba nějaká specifická inicializace, musíme ji provést v této metodě.
- Jak řešit **@Before** a **@After** společné pro podskupinu **@Test** metod?
  - ~~Duplikovat kód ve všech těchto metodách~~
  - Zvláštní "resource class"
  - Rozdělit testy do dvou tříd
  - Přechod na TestNG :)
- Konstruktor se vykonává pokaždé před **@Before**. Je v něm možné provést interní inicializaci proměnných atd., ale duplikuje metodu **@Before**. Není standardní řešení.

# Běh JUnit testů

- **Spouštění testů**
  - V JUnit sekvenčně za sebou, paralelní spouštění testů jde nepřímo
- **Nahlášení chyby**
  - Unit test nahlásí chybu, pokud skončí výjimkou

**Důsledky pro používání try-catch na cvičeních**

- **Garance pořadí spouštění testovacích tříd**:
  - Obecně není garantováno
  - Je možné přesně definovat pomocí TestSuite – viz cvičení
- **Garance pořadí spouštění @Test metod v rámci testovací třídy**:
  - Obecně není garantováno
  - Od JUnit 4.11 je možné definovat pomocí anotace ***@FixMethodOrder(MethodSorters.{SORTER})*** nad deklarací třídy unit testu

# Porovnávání výsledku

- Metody „**assert**“ třídy Assert
- Reference: http://junit.sourceforge.net/javadoc/org/junit/Assert.html

Obecné schéma:

- Konkrétní typ metody assert přetížený do několika variant
- Typické parametry:
  - Zpráva (verze se zprávou a bez)
  - Expected – očekávaný výsledek testu (**E**)
  - Actual – reálný výsledek testu (**A**)

  Nebo:
  - Condition – logická podmínka, která se testuje (**C**)

- Pokud srovnání nebo podmínka v metodě assert nejsou splněné, test skončí s chybou:
  - Je vyhozena výjimka a JUnit Framework tuto situaci vyhodnotí jako neúspěšný test

- Pozor na deprecated metody pro konkrétní verzi JUnit

# Porovnávání výsledku

Základní typy metod assert:

| Typ | Popis |
|---|---|
| assertEquals | • Porovnání dvou hodnot E a A |
| assertArrayEquals | • Porovnání dvou polí E a A |
| assertTrue | • Test, jestli podmínka C je splněná |
| assertFalse | • Test, jestli podmínka C není splněná |
| assertNull | • Test, jestli A je null |
| assertNotNull | • Test, jestli A není null |
| assertSame | • Porovnání, jestli objekty E a A jsou stejné (ve smyslu reference na objekt) |
| assertNotSame | • Porovnání, jestli objekty E a A nejsou stejné (ve smyslu reference na objekt) |
| assertThat | • Test, jestli A splňuje podmínku definovanou pomocí třídy org.hamcrest.Matcher (součást JUnit) |
| fail | • Test skončí s chybou |

**Další detaily týkající se syntaxe, organizace unit testů a test suites na cvičeních**

# Typy code coverage

Obecný vzorec:
**coverage = ( počet elementů pokrytých testy / počet elementů celkem ) * 100%**

**Line coverage (statement coverage)**

- Byl každý příkaz v kódu otestován?

**Branch coverage**

- Byla každá možná větev v rozhodovací struktuře kódu otestována?
- *Plus již probrané Test depth level N coverage*

**Function coverage**

- Byla každá metoda / funkce / procedura / podprogram otestována?

**State coverage**

- Byl každý stav ze stavového automatu otestován?
- *Plus již probrané N-switch coverage*

*Plus již probrané* ***CC, DC, C/DC, MC/DC, MCC****, přímo aplikovatelné na kód*

# Jak změřit code coverage?

Výběr ze základních frameworků pro měření code coverage:

**JaCoCo**

- pracuje online s bytecodem (za běhu)
- commandline obsluha, integrace pro Ant, Maven a Jenkins, pluginy do IDE
- http://www.eclemma.org/jacoco/trunk/index.html

**Cobertura**

- pracuje offline s bytecodem
- commandline obsluha, integrace pro Ant a Maven
- http://cobertura.github.io/cobertura/

**Clover**

- pracuje se zdrojovými kódy
- commandline obsluha, integrace pro Ant, Maven a Jenkins, pluginy do IDE
- https://www.atlassian.com/software/clover/overview

**EMMA**

- již se nevyvíjí, od 2005 nahrazeno projektem JaCoCo

# TestNG

# TestNG vs. JUnit

**V současné době je TestNG vhodnější, pokud Unit Test framework využíváme jako orchestrátor pro FE a integrační automatické testy.**

Klíčové funkce TestNG oproti JUnit:

- Individuální @Before a @After pro jednotlivé testy
- Vícenásobná hierarchie „folderů“ pro testy s vlastními @Before a @After – vhodné pro rozsáhlý set unit testů
- Lepší definice TestSuite – XML s více možnostmi strukturování
  - Více možností řízení sekvence testů než v případě JUnit
- Unifikace stylu práce s testovacími daty: DataProviders

**JavaDoc:**

http://testng.org/javadocs/index.html

# Instalace TestNG

Pokud není v IDE jako default (např. v NetBeans je, v Eclipse J2EE balíku není), nejlepší je přidat přes Maven:

V **pom.xml** přidat dependency:

```
<dependency>
  <groupId>org.testng</groupId>
  <artifactId>testng</artifactId>
  <version>6.1.1</version>
  <scope>test</scope>
</dependency>
```

# Zápis assertů

Operátor **assert**, nebo klasický styl z JUnit:

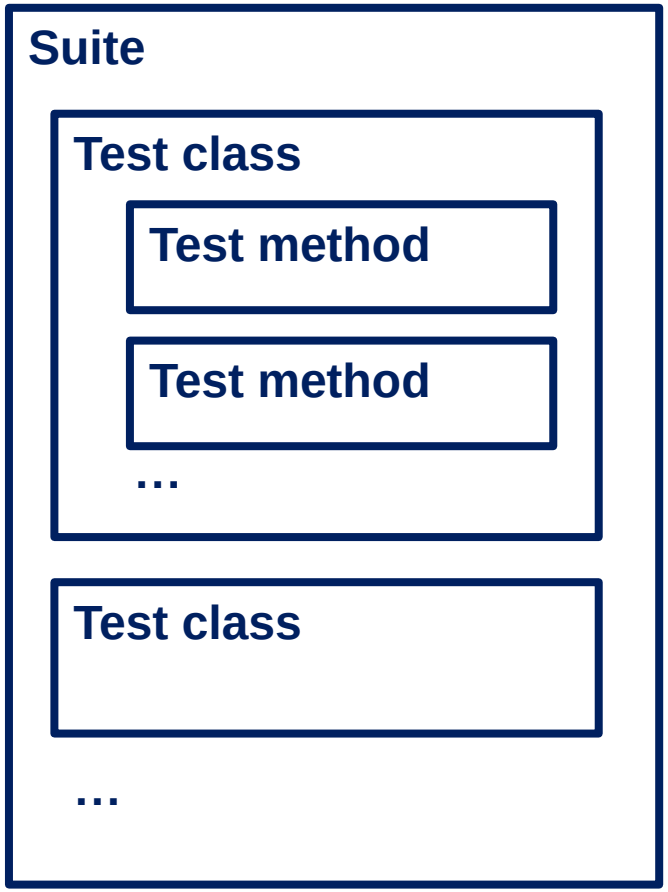
```
@Test
public void verifyLastName() {
  assert "Franta".equals(m_lastName) : "Expected name Franta, for"
   + m_lastName;
}
```

```
import static org.testng.Assert.*;
   // Nebo (pro složitější asserty z JUnit):
   // import static org.testng.AssertJUnit.*;

@Test
public void verify() {
  assertEquals("Franta", m_lastName);
}
```

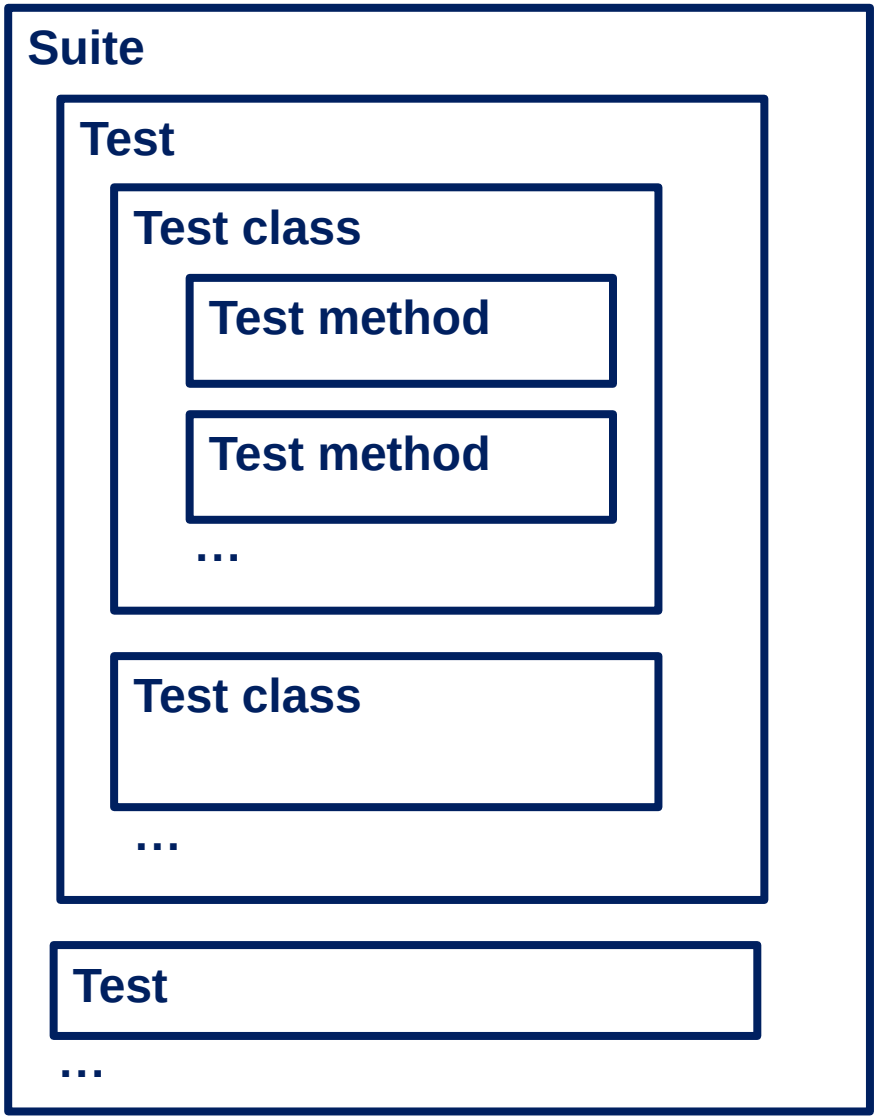
# Hierarchie testů

**JUnit:**




**TestNG:**

**+GROUPs**



# Test Suite

Na rozdíl od JUnit, TestSuite není Java class ale XML soubor:

```
<suite name="TestNG_trial">
    <test name="DummyTestSet1">
        <classes>
            <class name="testng_trial.DummyTestNGTest" />
            <class name="testng_trial.TestNGRunningSequenceTest" />
        </classes>
    </test>
</suite>

<suite name="TestNG_trial">
     <test name="testng_trial suite">
        <packages>
            <package name="testng_trial"/>
        </packages>
    </test>
</suite>
```

# Syntaxe a spouštěcí plán

**Konstruktor:**

- Na rozdíl od JUnit, konstruktor se nevykonává před každým spuštěním @Test, ale pro každou TestNG třídu jen jednou a to na začátku před spouštěním testů (ještě před @BeforeTest)

**Individuální Before a After pro jednotlivé testy:**

- Zařadím konkrétní test do skupiny pomocí @Test(groups={"grp1"})
- Specifikuji @Before pro skupinu: @BeforeGroups(groups={"grp1"})

# Spouštěcí plán

**všechny definované @BeforeSuite v TestNG testech, které jsou součástí test suite**

**všechny definované @BeforeTest v TestNG testech, které jsou zařazené v <test> tagu v test suite**

**CLASS 1:**

**@BeforeClass**

***** @BeforeGroups**

**@BeforeMethod**

**test1**

**@AfterMethod**

***** @AfterGroups**

**@BeforeMethod**

**test2**

**@AfterMethod**

**@BeforeMethod**

**test3**

**@AfterMethod**

**@AfterClass**

**CLASS 2:**

**Stejné schéma jako Class1**

**… další testovací třídy**

**všechny definované @AfterTest v TestNG testech, které jsou zařazené v <test> tagu v test suite**

**všechny definované @AfterSuite v TestNG testech, které jsou součástí test suite**

# Koexistence TestNG a JUnit (NetBeans)



**Obecné spuštění testů (funkce Test project):**

- TestNG spustí i JUnit testy, při testu celého projektu nejdřív JUnit testy a potom TestNG testy
- Funguje chaoticky, někdy spustí jen první JUnit test

**Spouštění TestNG testů z JUnit test suite:**

- NetBeans vytvoří JUnit TestSuite pro TestNG soubory, ale při exekuci toho JUnit TestSuite se v něm TestNG testy nespustí

**Spouštění JUnit testů z TestNG test suite:**

- Pokud se v TestNG test suite použije **<packages>** na celý package a jsou v něm i JUnit testy, ty se explicitně nespustí.
- Jejich spuštění je potřeba explicitně zajistit pomocí **<test name="JUnit all package execution" junit="true">**.

- Definice na samotnou třídu **<class name="testng_trial.DummyJUnitTest" junit="true" />** se ignoruje.

# Data providers

- http://testng.org/doc/documentation-main.html#parameters-dataproviders
- Data providery a test metody mohou být obecně mapovány M:N

**Definice ve stejné třídě s testy:**

```
//This method will provide data to any test method that declares that its Data
   Provider is named "test1"
@DataProvider(name = "test1")
public Object[][] createData1() {
 return new Object[][] {
   { "Cedric", new Integer(36) },
   { "Anne", new Integer(37)},
 };  // ZDE LOAD Z GRIDU / DB
}

//This test method declares that its data should be supplied by the Data
   Provider named "test1"
@Test(dataProvider = "test1")
public void verifyData1(String n1, Integer n2) {
 System.out.println(n1 + " " + n2);
}
```

# Data providers

**Definice mimo třídu s testy:**

```
public class StaticProvider {
  @DataProvider(name = "create")
  public static Object[][] createData() {
    return new Object[][] {
      new Object[] { new Integer(42) }
    }
  }
}

public class MyTest {
  @Test(dataProvider = "create", dataProviderClass = StaticProvider.class)
  public void test(Integer n) {
    // ...
  }
}
```

# Externí test report (kromě IDE)

- TestNG defaultně ukládá report v HTML do *[project_folder]*/test-output

Test results

1 suite, 3 failed tests

All suites

Default suite

**Info**

- C:\Users\mibure\AppData\Local\Temp\testng-eclipse-561804759 \testng-customsuite.xml
- 1 test
- 3 groups
- Times
- Reporter output
- Ignored methods
- Chronological view

**Results**

- 8 methods, 3 failed, 5 passed
- Failed methods (hide)
  - dummyTestNGTest1(2, b)
  - dummyTestNGTest1(1, a)
  - dummyTestNGTest2
- Passed methods (show)

testng_tests.DummyTestNGTest

```
dummyTestNGTest1 (2, b)
   java.lang.AssertionError: Dummy test 1 - no imple
            at org.testng.Assert.fail(Assert.java:94)
            at testng_tests.DummyTestNGTest.dummyTest
            at sun.reflect.NativeMethodAccessorImpl.
            at sun.reflect.NativeMethodAccessorImpl.
            at sun.reflect.DelegatingMethodAccessorIm
            at java.lang.reflect.Method.invoke(Unknow
            at org.testng.internal.MethodInvocationHe
            at org.testng.internal.Invoker.invokeMeth
            at org.testng.internal.Invoker.invokeTest
            at org.testng.internal.Invoker.invokeTest
            at org.testng.internal.TestMethodWorker.
            at org.testng.internal.TestMethodWorker.
            at org.testng.TestRunner.privateRun(TestR
            at org.testng.TestRunner.run(TestRunner.
            at org.testng.SuiteRunner.runTest(SuiteRu
            at org.testng.SuiteRunner.runSequentially
            at org.testng.SuiteRunner.privateRun(Suit
            at org.testng.SuiteRunner.run(SuiteRunner
            at org.testng.SuiteRunnerWorker.runSuite
            at org.testng.SuiteRunnerWorker.run(Suite
            at org.testng.TestNG.runSuitesSequential
            at org.testng.TestNG.runSuitesLocally(Tes
            at org.testng.TestNG.run(TestNG.java:1057
```

# Základy efektivního návrhu

# Doporučená pravidla

- **Unit test nezávislý na dalších testech**:
  - Pokud jde, běh konkrétního unit testu nesmí být prerekvizita pro jiný test
  - Snažte se o minimalizaci potřebných vstupních podmínek (preconditions). Někde se jim ale nevyhnete.
- **Jedna funkcionalita ~ jeden unit test**
  - Ušetří režii při aktualizaci unit testů, pokud se mění funkcionalita
  - Obecný problém údržby testovacích skriptů
- **Přiměřený počet assertů**: víc assertů, které testují stejnou situaci nezvýší pokrytí, pouze zvýší režii na údržbu a sníží přehlednost výsledků testů
  - Při návrhu assertů aplikujte standardní test design techniky známé z manuálního testování
- **Nepište unit testy pro kontrolu konfigurací** nebo nastavení, pouze kódu. Konfigurace nebo kontext se může měnit, testy pak zbytečně padají
  - Je vhodné vytvořit zvláštní test set pro výchozí kontrolu konfigurace, pokud tato kontrola proběhne úspěšně, pak teprve spusťte další testy

# Doporučená pravidla

- **Srozumitelné názvy metod unit testů**
  - Název unit testů vyjadřuje, co testuje, včetně jednoduchého popisu průběhu
  - Konzistentní jmenná konvence
  - Příklad:
    - test*[metoda/funkce]_[testová situace]_[očekávaný výsledek]*()
  - Názvy metod založené pouze na testované funkci mohou být pro více testovacích situací málo přehledné a složité na údržbu
- **Nesmíme měnit testovanou aplikaci**
- Jednotný zdroj parametrizace testovacích dat – pokud je to možné, nemíchejte dohromady různé zdroje (různé DB, různé konfigurační soubory)

- Kde je to vhodné, **strukturujte testy podle objektové struktury kódu** (viz dělení dále)
  - Třídě odpovídá testovací třída
  - Metodě odpovídá testovací metoda

**Kdy taková struktura vhodná není?**

# Testdesign pro JUnit

Pro efektivitu unit testů platí velmi analogická pravidla jako pro manuální testování. V důsledku – i zde vyvoláváme vykonání částí kódu, pouze jiným způsobem. **Používejte vhodným způsobem test design techniky** (viz předchozí přednášky).

Toto platí i pro využití unit testů pro konstrukci kódu (Test Driven Development, viz dále)

- **Klasické jednotkové testy**
  - Krok 1: určení bloku kódu / funkcionality pro test:
    - Testujeme public metody. Jak s private metodami? Viz. cvičení
  - Krok 2: analýza struktury v bloku vybraném pro unit test a výběr test design techniky:
    - Kombinace vstupů (parametry) a očekávaného výsledku

      → Techniky pro pokrytí vstupů, Elementary comparison test, …

**Častý přístup – testy se píší pro metody, delší sekvence se nevyvolávají. Proč?**
**Jak je to z pohledu údržby a aktualizace unit testů?**

# Testdesign pro JUnit

- **Procesní/integrační testy**
  - *Poznámka: Podle délky procesu a toho, jestli pracují s interními nebo externími rozhraními by se daly zařadit spíše do integračních testů. Zde je máme v tématu unit testů, protože (1) jdou implementovat unit test frameworkem a (2) na integrační testy nemáme samostatnou přednášku :)*

- Krok 1: určení bloku kódu / funkcionality pro test:
  - Sekvence více metod (část procesu)
  - Blízká API – typicky mezi moduly systému
  - Vzdálená API – propojují systémy navzájem
  - E2E test – je vhodný?
- Krok 2: analýza struktury v bloku vybraném pro unit test a výběr test design techniky:
  - Sekvence metod za sebou, zpracování má skončit očekávaným výsledkem, v průběhu kontrolujeme mezivýsledek

    → Procesní test, Test životního cyklu dat, …

# Testdesign pro JUnit

- **Procesní/integrační testy**

  - „Problém předpokladu“ pro tento typ testu: Vykoná reálný uživatel přes uživatelské rozhraní aplikace nebo voláním služeb systému skutečně vaši předpokládanou testovou sekvenci (nakódovanou v procesním/ integračním testu)?
    - Jak se tomu při návrhu testu co nejvíce přiblížit?

# Dobrý návrh testů



**Klasické jednotkové testy, Integrační a FE testy**

- Arrange-act-assert sktruktua
- Čitelný kód
- Srozumitelné jmenné konvence
- Konkrétní assert zprávy → snadnější reportování
- Konfigurace mimo sadu testů

**Klasické jednotkové testy navíc:**

- Nezávislost jednotkových testů
- Granularita

# Code smells

**Vyhněte se následujícím negativním vzorům (antipatterns):**

- Weak tests (not likely to fail)
  - Wrong try-catch inside
- Overprotective tests (failing also for irrelevant causes)
- Readability
  - Unclear assertions
  - Hyperassertions
  - Too much code extra to the test making it unclear, long code
  - Using magic numbers instead of self-explanatory constants
  - Too extensive set-up code
- Granularity
  - @Test checking multiple things

# Efektivní použití unit testů

**Názor 1:**

- Alespoň nějaké unit testy jsou lepší než žádné

**Za jaké podmínky?**

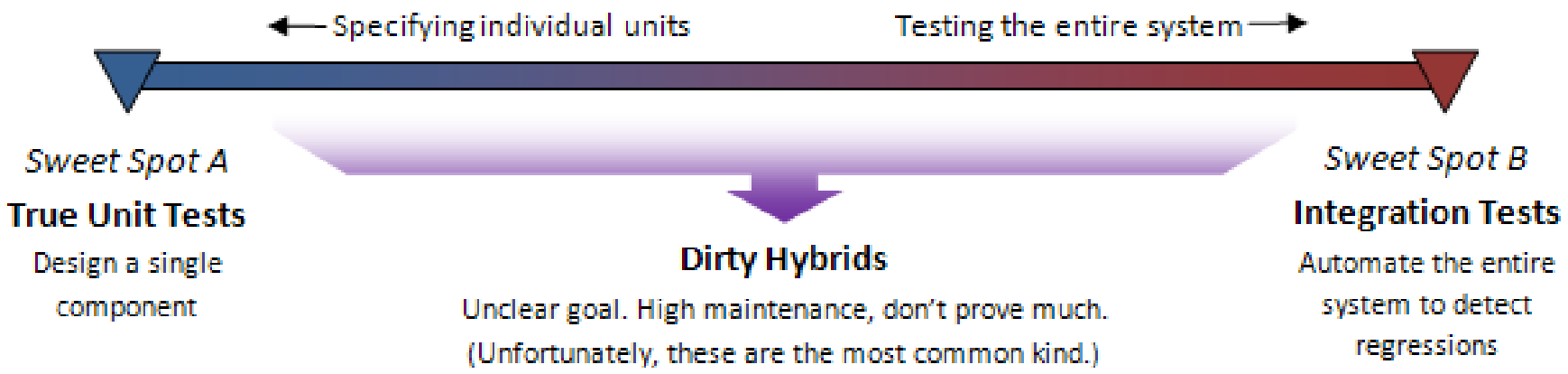
**Názor 2:**

- Čím víc unit testů tím lépe

**Opravdu? Jaký je vztah k test coverage:**
**Branch nebo line coverage**
**Obecně - potenciál unit testů odhalit chyby**

# Efektivní použití unit testů

**Názor 3:**




Zdroj: http://blog.stevensanderson.com/2009/08/24/writing-great-unit-tests-best-and-worst-practises/

**Unit testy jen jako prostředek TDD nebo je rozšířit na kompletní integrační testy?**

# Vývojové metodiky využívající jednotkové testování

# Test driven development (TDD)



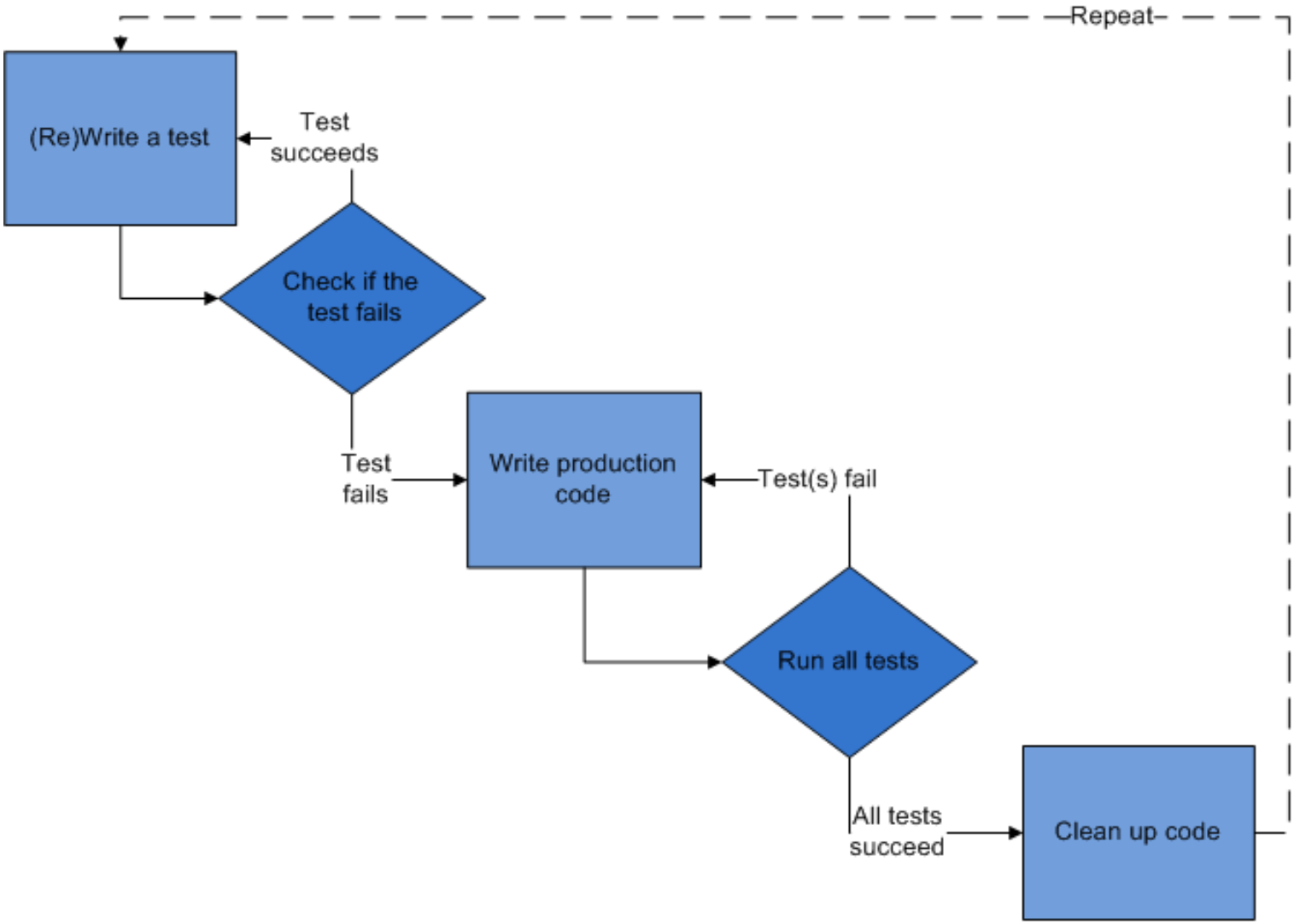
- Agilní metodika vývoje SW
- Jedna z komponent metodiky Extreme programming

<u>Základní postup TDD:</u>

- Vytvoření testu podle očekávaného výsledku funkcionality
  - Nový test neprojde, všechny ostatní předchozí testy mají projít
- Vytvoření první verze kódu s funkcionalitou
  - Všechny předchozí testy mají projít
- Refaktoruj kód
  - Všechny předchozí testy mají projít
- Přidej další funkcionalitu (cyklus se opakuje)

# Test driven development (TDD)

Schéma postupu TDD:




Zdroj: http://en.wikipedia.org/wiki/Test-driven_development

# Test driven development (TDD)



Náhrada (v danou fázi vývoje) neexistujících částí kódu (podle ISTQB):

**Stub**

- Nahrazuje volanou komponentu
- Simuluje návrat / vyvolání očekávaných výsledků podle konkrétního vstupu

**Driver**

- Komponenta nebo nástroj, která simuluje komponentu zodpovědnou za řízení a/nebo volání jiných komponent

**Simulator** (Emulator)

- Zařízení, program, nebo systém, který se chová jako testovaný systém, pokud dostane konkrétní sadu potřebných vstupů

Koncepty jsou stejné i pro klasický vývoj a integrační testování

# Výhody a limity TDD

| Výhody | Limity |
| --- | --- |
| • Rovnou vytváříme unit testy v souladu se specifikací → minimalizace regrese, všechny výhody unit testů<br><br>• Agilní přístup k vytváření kódu, rovnou verifikuji, neodkládám „sestavování“ neverifikovaných částí na později | • Přechod na tento agilní způsob znamená poměrně velkou změnu stylu práce. Nestačí rozhodnutí „a nyní to děláme agilně“.<br><br>• Problém se specifikací očekávání a testů na začátku. Může být tendence rychle nadefinovat testy a pak kódovat<br>→ účinnost testů klesá<br>→ iluze o dobré test coverage |

- Problém očekávání: Slouží TDD pro specifikaci a konstrukci kódu, nebo má zároveň plnit i roli testovací metodiky?

# Acceptance TDD (ATDD)

- Vývojový tým diskutuje a stanovuje akceptační kritéria s klientem
- Podle nich vytvoří set akceptačních unit testů, než začne vývoj

- Akceptační kritéria je zde potřeba chápat jako specifikaci chování

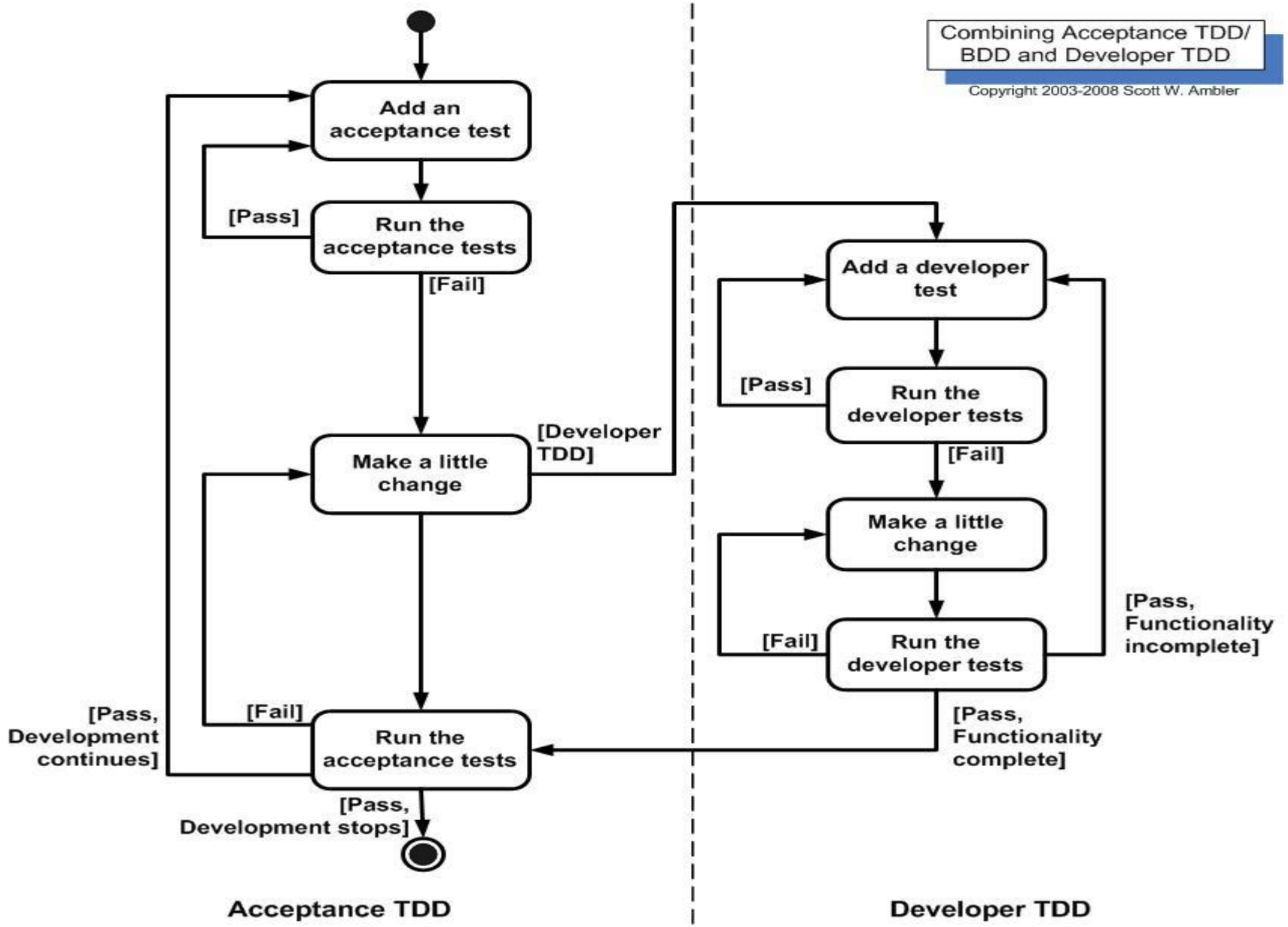
- Velmi vhodné pro agilní vývoj
- Použitelné i pro klasické metodiky vývoje SW

- Klienta držíme v kontaktu s vývojem produktu (viz V a W-model)

- Jistá analogie s konceptem prototypování

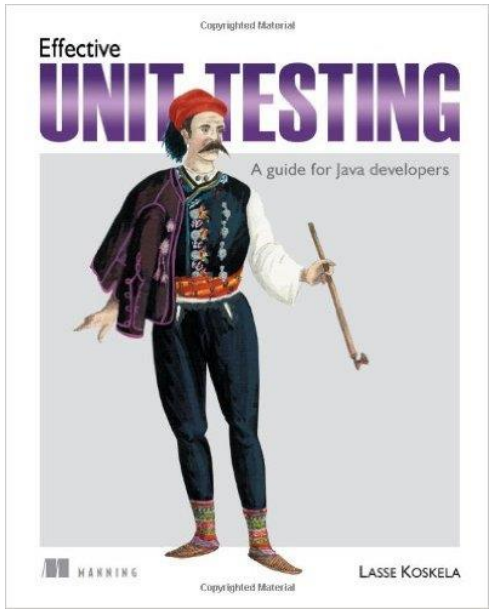
# Schéma ATDD

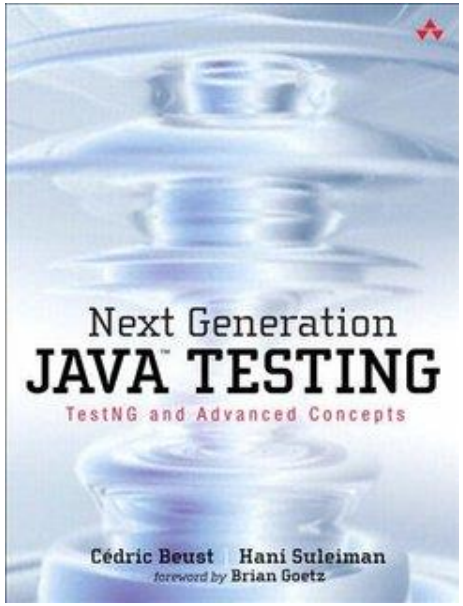
Zdroj: http://www.agiledata.org/essays/tdd.html



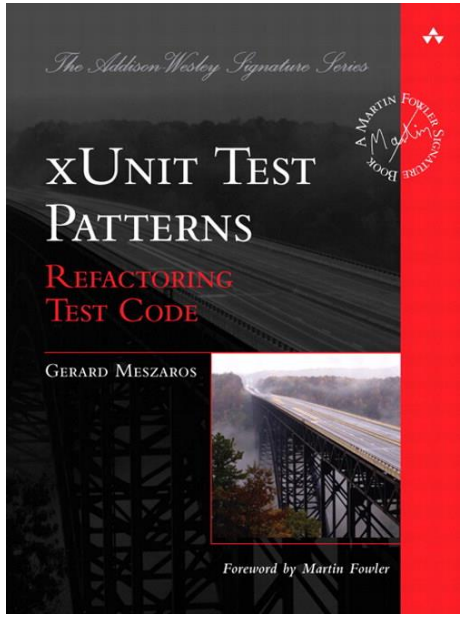
# Další literatura

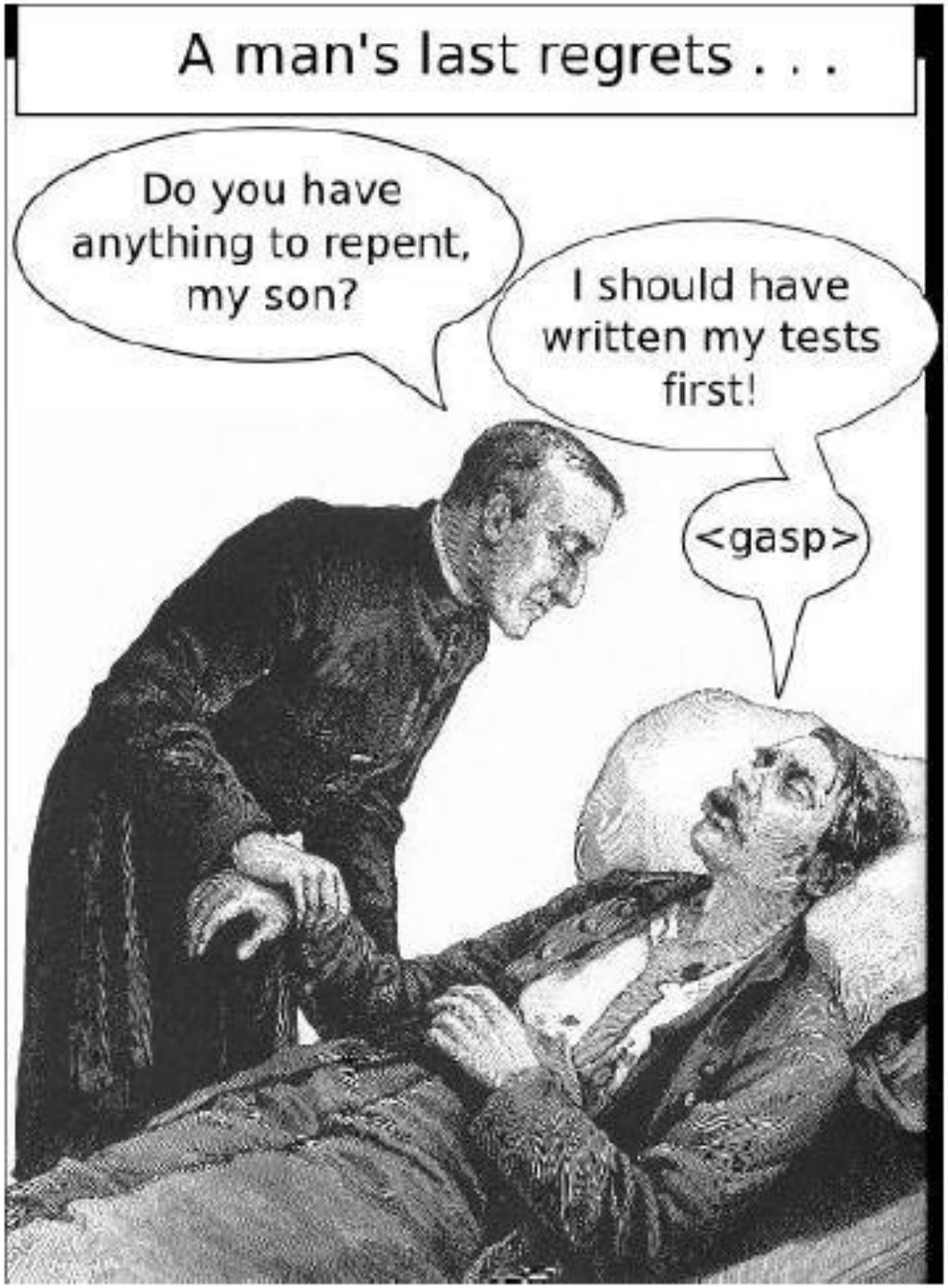
**Pro středně pokročilé:**




**Pro pokročilé:**




A man's last regrets . . .
Do you have anything to repent, my son?
I should have written my tests first!
<gasp>

**Thank you for attention**

buresm3@fel.cvut.cz

# Zdroje

ISTQB Certified tester Foundation level syllabus, release 2011
http://www.istqb.org/downloads/viewcategory/16.html

ISTQB Glossary of testing terms, version 2.2
http://www.istqb.org/downloads/viewcategory/20.html

Doporučení pro unit testy – volně podle:
http://blog.stevensanderson.com/2009/08/24/writing-great-unit-tests-best-and-worst-practises/

Code coverage measurement tools pro Javu:
http://en.wikipedia.org/wiki/Java_Code_Coverage_Tools

Test Driven Development:
Kent Beck, Test Driven Development: By Example. Addison-Wesley, 2003